# Estratégia da Coleta do TCCELTA

18 de outubro de 2026

## 1 Introdução

O objetivo deste documento é discutir a estratégia de coleta do Dongle OBD-II do TCCELTA. Anteriormente, o Dongle era um dispositivo passivo, que simplesmente respondia às requisições do aplicativo. Com a mudança do foco para coleta de telemetria, o Dongle passa a proativamente requisitar e registrar os dados do CAN. Isto gera dificuldades que serão apresentadas aqui, bem como a solução proposta.

O Dongle estará conectado ao barramento CAN e realizará leituras de PIDs. Essas leituras serão agrupadas em **amostras**, e enriquecidas com o timestamp da coleta e com informações de GPS (latitude, longitude e altitude). As amostras coletadas devem ser enviadas ao servidor por intermédio de um aplicativo, que pode não ser confiável. O servidor que deve ser capaz de verificar a autenticidade dos dados, ou seja, que eles foram gerados pelo Dongle.

Há uma série de dúvidas que surgem em relação à coleta. O que aconteceria no caso do Dongle funcionar sem conexão com um aplicativo, e portanto, sem Internet? O Dongle não possuirá módulo de GPS, e nem conexão direta à Internet; assim, como que as amostras poderão conter dados de GPS e timestamp? Qual será a frequência de coleta dos dados? Qual será o tamanho de cada amostra? Quanto tempo o Dongle consegue funcionar antes de ocorrer perda de dados por não possuir mais armazenamento livre? Como detectar perda de dados?

Há, ainda, certas dúvidas acerca do barramento CAN. Não temos carro para testá-lo, e ainda que tivéssemos, não poderíamos supor que seu funcionamento seria representativo de todos os modelos existentes. Logo, para projetar a comunicação, devemos nos ater à Teoria, com o suporte das especificações.

Será apresentada uma proposta de implementação para a coleta do Dongle. Existem quatro principais problemas que guiam toda a implementação. Eles serão apresentados a seguir.

### 1.1 Uma coisa de cada vez

A ISO 15765-4 define que a taxa de transferência do barramento CAN é, no máximo, 500kbps. Além disso, as leituras de PIDs provém de requisições OBD-II. Em teoria,

cada requisição pode pedir até 6 PIDs de uma só vez; no entanto, não encontrei documentação o suficiente sobre como isso funciona.

Se não há garantia de que pedir vários PIDs em uma única requisição terá uma resposta adequada, uma alternativa seria contar com a concorrência. Ou seja: colocar sucessivamente no barramento uma requisição para cada PID desejado, e depois escutar e coletar as respostas conforme fossem aparecendo. No entanto, também não há garantia alguma que isto funcione. Algumas ECUs podem simplesmente atender somente a requisição mais recente.

Logo, para evitar retrabalhos, o caminho que se apresenta é o de assumir o pior caso, ou seja, que não será possível contar nem com a leitura de múltiplos PIDs em uma única requisição, nem com o atendimento de várias requisições concorrentes. A comunicação modelada será, portanto, estritamente sequencial, representando assim o pior desempenho possível e garantindo que o Dongle funcionará mesmo neste caso. A negação de qualquer uma dessas hipóteses (não-concorrência e somente pedidos únicos) representaria uma boa notícia e uma oportunidade de melhorar o desempenho do projeto em uma futura evolução.

Ter a esperança de uma possível boa notícia no futuro não alivia a má notícia no presente. Estar restrito a uma comunicação sequencial representa um grande desafio. No datasheet do ELM327 encontra-se a informação de que, para cada leitura, ele pode esperar até 200ms antes de declarar um timeout. Supondo que esse seja o tempo de resposta médio para as requisições do nosso projeto, isso significaria a leitura de somente 5 PIDs por segundo, com grande diferença temporal entre a aquisição dos PIDs!

Quanto mais PIDs desejarmos adicionar às amostras, maior será o tempo necessário para gerar uma amostra, e maior será o período representado por uma coleta. Isto pode apresentar severas distorções. Por exemplo, durante uma frenagem, o RPM pode ser lido no início do ciclo, ainda elevado, e a velocidade no final, já reduzida. Como os dois valores correspondem a instantes diferentes, a razão entre velocidade e RPM fica inconsistente, o que poderia ser interpretado erroneamente como uma troca de marcha. Ademais, neste cenário, qual seria um timestamp adequado? O da primeira coleta? O da última? Um para cada PID coletado?

Estes problemas são consequência do fato de que a arquitetura parece inadequada à solução. Os sensores não estão conectados diretamente no nosso coletor; ele deve acessá-los por intermédio de um barramento no qual seus pedidos são de baixa prioridade. Isto pode funcionar para diagnóstico; para telemetria, implica em uma grande complexidade. Devemos fazer milagre.

## 1.2 Uma questão de tempo

O Dongle não possuirá acesso direto à Internet, nem possuirá módulo RTC. Isto por si só já representa uma limitação ao projeto, que deveremos justificar perante à banca. Mas, supondo a ausência, precisamos lidar com ela.

Poderíamos simplesmente aceitar o timestamp que o aplicativo enviar. Mas já esta-

belecemos que ele não é confiável. Como contornar isto?

Infelizmente, sem um RTC, há somente soluções parciais e sujeitas a serem prejudicadas pelo intermediário não confiável. É possível garantir a preservação da ordem correta dos dados e do intervalo de tempo entre eles; mas o instante de tempo absoluto, não. Eu não considero este um problema tão grave; algo similar poderia acontecer mesmo se tivéssemos um RTC, caso um atacante descarregasse sua bateria ou o reiniciasse de alguma forma.

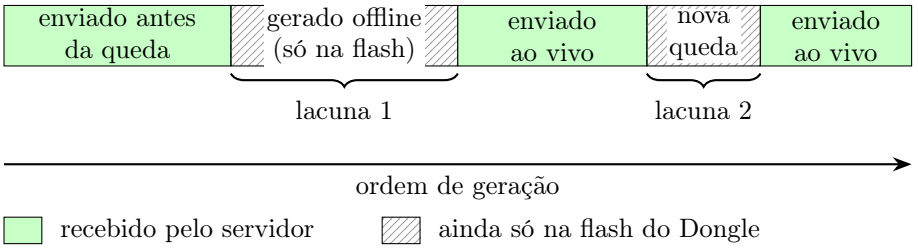
A garantia do ordenamento e do intervalo de tempo entre os dados será discutida na proposta de implementação.

## 1.3 Desconectado

Se o Dongle não estiver conectado a um aplicativo, ou se o aplicativo estiver sem acesso à Internet, os dados não serão enviados ao servidor. Ou seja, para evitar perda de dados, será necessário armazená-los localmente até que haja conectividade.

Isto gera problemas de sincronização. É necessário saber quais dados não foram recebidos pelo servidor. E, se o armazenamento encher, haverá perda de dados. É necessário detectar e reportar esta perda.

Além disso, no cenário em que há conexão com a Internet, pode haver dados antigos no armazenamento local aguardando envio enquanto novos dados continuam sendo gerados. Se for priorizado o envio dos dados recentes, o servidor deve ser capaz de detectar a lacuna entre eles e os dados antigos ainda não recebidos. E, se a conectividade for perdida novamente nesse cenário, surgirá uma segunda lacuna, ou potencialmente várias, tornando o controle do que já foi recebido cada vez mais complexo.




## 1.4 Cachorro com dois donos morre de fome

Como foi discutido na Subseção 1.1, a taxa de leitura de PIDs é potencialmente baixa, o que ocasiona problemas na geração de amostras.

Porém, uma das premissas do projeto é possuir um aplicativo que permita a visualização dos dados em tempo real. O aplicativo irá periodicamente solicitar a leitura PIDs, e utilizará as respostas para gerar suas visualizações.

Assim, além de fazer as requisições necessárias para gerar amostras, o Dongle também atender prontamente aos pedidos arbitrários do aplicativo, sem que uma tarefa prejudique a outra!

### 1.5 Vai se criando um clima terrível...

Os problemas apresentados não são simples e não temos muito tempo para endereçá-los completamente. Então simplesmente não iremos. Optaremos pela simplificação das coisas, e o que ficar descoberto será documentado como limitação. A prioridade é ter algo com alguma garantia que funcionará caso tenhamos a possibilidade de conectar o Dongle a um carro real.

Uma solução é apresentada a seguir.

## 2 Uma possível solução

### 2.1 Agradando a dois senhores

O Dongle contará com dois modos de operação: Coletor e Diagnóstico. O modo Diagnóstico corresponde ao que já temos e dá suporte a um aplicativo de leitura em tempo real. Sua segurança é dada pelo BLE Secure. Simples assim.

O modo Coletor corresponde à geração de telemetria. O pressuposto deste modo é que será definida uma lista de PIDs, que serão consultados de modo a gerar as amostras. Durante este modo de operação, o Dongle responderá somente a pedidos do app que forem referentes à leitura de PIDs contidos na lista de PIDs coletados para telemetria. Qualquer outro pedido não será atendido, excetuando-se o pedido de troca de modo.

Em uma versão comercial, o modo de operação padrão do Dongle após um reinicio seria o modo Coletor, e um pedido de troca de modo só deveria ser aceito pelo Dongle se fosse acompanhado do MAC do servidor e de um número X, com teto máximo Y. Após X minutos ou após um reinicio, o dongle voltaria ao modo Coletor. Isto permitiria a utilização avançada de diagnóstico (e portanto interrupção na coleta de telemetria) somente mediante prévia autorização do servidor, e com tempo limite. No entanto, para fins de facilitar demonstração do projeto, simplesmente sempre aceitaremos um pedido de troca de modo. Esta decisão será documentada.

Esta abordagem endereça o problema dos Dois Senhores, apresentado na Subseção 1.4. Cada modo de operação prioriza uma estratégia de leitura e permite simplificar o código e desafogar o barramento.

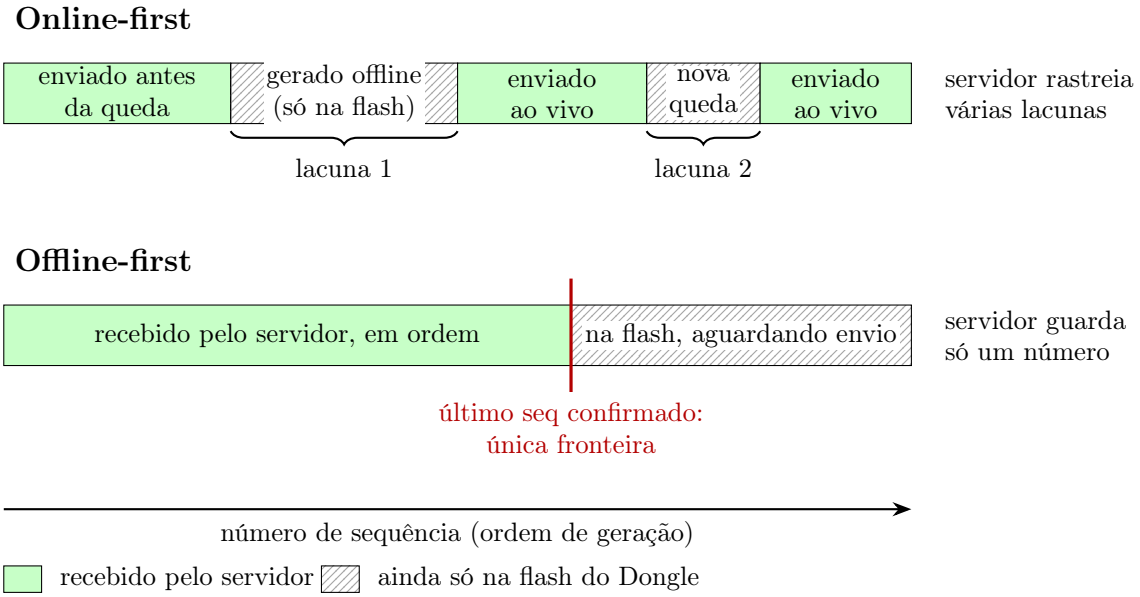
### 2.2 Ordem e Progresso

O Dongle possuirá um número de sequência monotônico crescente, chamado **seq**. Ao gerar uma amostra, atribuirá este número a ela, e depois o incrementará. Este número será armazenado na memória não-volátil para sobreviver a reinicializações.

O servidor irá armazenar o maior número recebido sem lacunas, que será enviado como ACK ao Dongle.

Isto permite a utilização da estratégia offline-first write-through. Dados coletados são escritos na memória local. Quando há conectividade, uma tarefa acorda e lê os

dados em lotes, a partir do número recebido no último ACK do servidor.




Isso força todos os dados a passarem pelo armazenamento local, o que atrasa sua chegada ao servidor. Parte do atraso vem do próprio caminho pela memória não volátil, já que um dado só é enviado depois que o bloco que o contém é fechado e gravado. A maior parte, porém, ocorre quando há dados antigos acumulados: os novos só são enviados depois que o envio dos antigos alcança a fronteira dos dados recém-gerados.

Considerando:

- $r$: taxa de geração de dados, em B/s
- $C$: vazão efetiva de envio até servidor, passando por BLE, aplicativo e Internet, em B/s
- $t_{\text{off}}$: tempo que o Dongle passou sem conexão

Durante o tempo offline, acumulam-se $B = r \cdot t_{\text{off}}$ bytes. Quando a conexão volta, o envio escoa a uma taxa $C$, mas novos dados continuam chegando a uma taxa $r$. Então, o acúmulo diminui a uma taxa $C - r$ e o tempo para a fronteira chegar até os novos dados em "tempo real"é:

$$t_c = \frac{B}{C - r} = \frac{r\, t_{\text{off}}}{C - r} \tag{1}$$

Uma estimativa conservadora é de cerca de 1 minuto a cada hora de coleta ininterrupta offline. Naturalmente, no TCC justificaremos a estimativa e podemos até calcular uma mais embasada utilizando medidas reais.

A abordagem offline-first simplifica muito o código e será a adotada. Isto endereça a questão da conectividade levantada na Subseção 1.3.

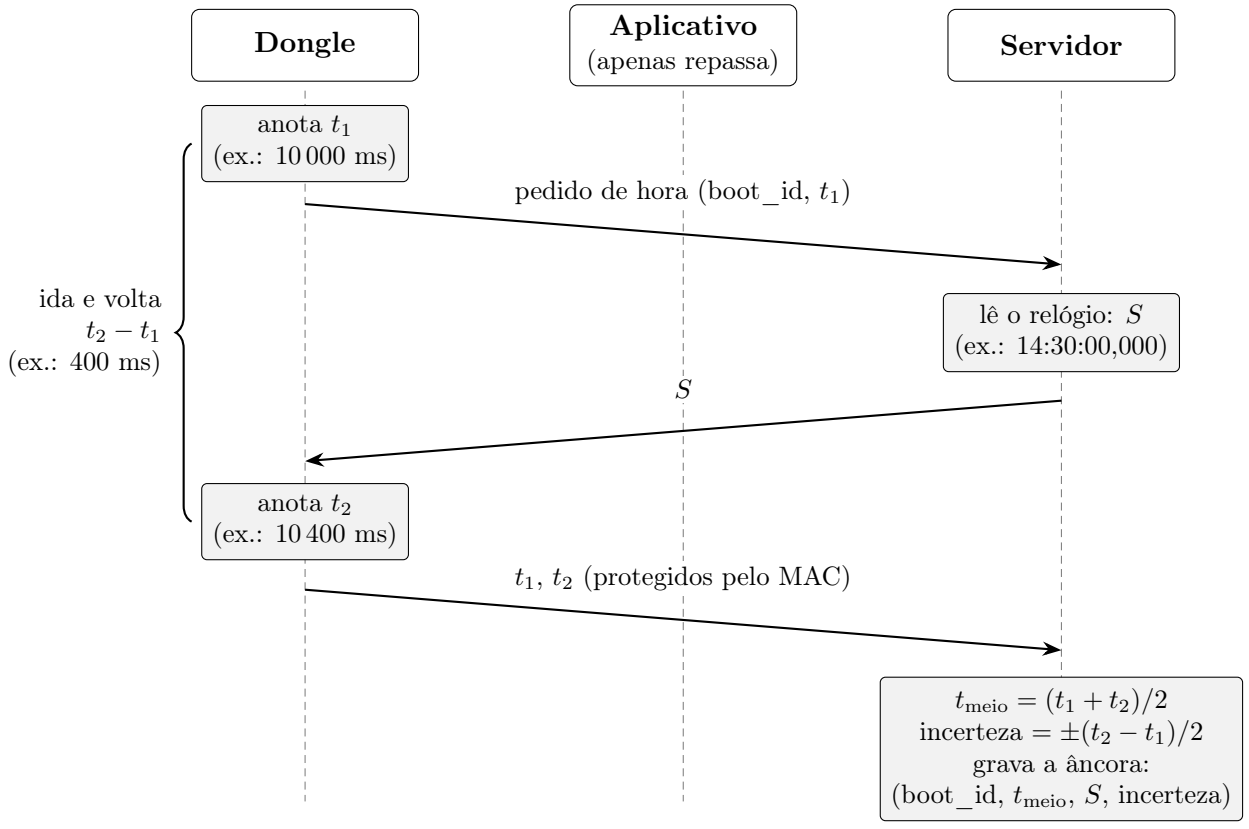
## 2.3 Sincronizando os Relógios

Não é o escopo deste documento discutir segurança. A premissa é que, na presença de Internet, o Dongle irá autenticar-se junto ao servidor, e irá adquirir uma chave de sessão. Ela será utilizada para gerar o MAC dos dados enviado, mas também pode ser utilizada para validar dados provenientes do servidor. Isto será útil no contexto a seguir.

Dado que o Dongle não tem relógio, ele irá proceder da seguinte forma. O Dongle manterá um número interno chamado de Boot Number. A cada reinicialização e portanto novo boot (que pode ser ocasionado por desconexão, deep sleep, entre outros), seu timer interno, que conta a quantidade de milissegundos desde o boot, reseta. Ao realizar a primeira coleta de um novo boot, irá gerar um novo Boot Number a partir do Boot Number anterior mais 1.

Todos os dados gerados serão enriquecidos com o Boot Number atual e com a quantidade de ms desde o boot. Com isso, junto com o **seq** citado na Subseção 2.2, garante-se o ordenamento e o intervalo de tempo entre os dados!

Para se obter o timestamp absoluto, é possível utilizar o Algoritmo de Cristian, que permite a sincronização de relógios de sistemas distribuídos.




Em resumo, trocam-se mensagens. O servidor recebe a leitura do relógio interno do Dongle duas vezes, e, com base na demora entre as requisições, estima a qual horário cada leitura do relógio do dongle daquele Boot Number corresponde, atribuindo uma incerteza que cresce com a demora entre as requisições. Quanto mais rápida a troca, menor a incerteza.

Na figura acima, a troca levou 400 ms, mas não dá para saber exatamente quando o servidor leu o relógio dentro desse intervalo. A melhor aposta é o meio: no instante t = 10 200 ms do Dongle, eram 14:30:00 no servidor. O erro máximo é metade da ida e volta, ±200 ms.

Com isso, o servidor passa a armazenar a correspondência Tempo Dongle <-> Tempo real, para aquele Boot Number. Assim, o servidor consegue atribuir um horário a todos os dados referentes ao Boot Number!

Um problema é que a comunicação sempre passa pelo App. Se este não for idôneo, pode atrasar as mensagens de propósito, aumentando a incerteza. Ou pode também simplesmente nunca repassar as mensagens. Nesses casos, assim como nos casos em que nunca há conexão com App ou Internet durante um boot, os dados nunca serão localizados no tempo pelo servidor, que tomará conhecimento somente da ordem e do intervalo de tempo entre dados de um mesmo Boot Number.

Isto endereça a questão de tempo levantada na Subseção 1.2

## 2.4 A hora da colheita (ou da coleta)

### 2.4.1 O que queremos

Inicialmente, é necessário definir os PIDs desejados. Proponho os seguinte 7:

Tabela 1: PIDs coletados pelo Dongle.

| PID | Bytes | Descrição |
|---|---|---|
| `04` | 1 | Carga calculada do motor (%) |
| `05` | 1 | Temperatura do líquido de arrefecimento (°C) |
| `0C` | 2 | Rotação do motor (RPM) |
| `0D` | 1 | Velocidade do veículo (km/h) |
| `10` | 2 | Vazão mássica de ar, MAF (g/s) |
| `2F` | 1 | Nível de combustível (%) |
| `11 / 45 / 49` | 1 | Posição do Pedal de acelerador (%) |
| Total | 9 | |

Eles devem ser o suficiente para realizar análises interessantes como: distribuição de RPM, velocidade, e posição do pedal; agressividade na condução; comportamento de troca de marchas; estimativa do consumo de combustível; detecção de Engine Lugging; etc.

Uma reflexão importante é que a frequência de coleta desses PIDs pode ser diferente! RPM e Velocidade mudam rapidamente; logo, é desejável que sejam amostrados com alta frequência; mais ainda, como um ponto de interesse é observar a relação entre eles para definir a marcha atual, também é desejável que sejam amostrados um após o outro.

Já Temperatura do Líquido de Arrefecimento e Nível de Combustível podem ser amostrados muito menos frequentemente, pois mudam muito menos rapidamente.

### 2.4.2 Leituras

Conforme a Subseção 1.1, dado que somente um PID será lido por vez, o tempo entre eles pode ser grande, o que prejudica a precisão de uma amostra, que deve esperar a leitura de todos os PIDs para ser gerada.

Para resolver isso, o Dongle adotará um comportamento simplificado de geração de leituras individuais, e não de amostras. Cada leitura conterá o valor de um PID específico, bem como o instante de tempo e a localização do GPS. O processamento delas ficará a cargo do Servidor.

Cada PID a ser coletado corresponde a uma linha de uma tabela mantida em memória, construída segundo o modelo de *Current Value Table* (CVT). Além do valor mais recente e do instante em que foi recebido, cada linha guarda as informações de escalonamento do PID, conforme a Tabela 2. A tabela é, assim, o ponto central da aquisição: o escalonador a consulta para decidir qual PID ler em seguida e a atualiza a cada resposta, enquanto o aplicativo apenas lê os valores, sem gerar novas requisições no barramento.

Tabela 2: Campos de cada linha da CVT.

| Campo | Uso | Descrição |
|---|---|---|
| PID | — | Identificador do parâmetro |
| Período | Escalonamento | Intervalo desejado entre leituras |
| Próxima leitura | Escalonamento | Instante em que o PID volta a vencer |
| Valor | Valor | Último valor recebido |
| Atualizado em | Valor | Instante da última resposta válida |

O fluxo é então o seguinte:

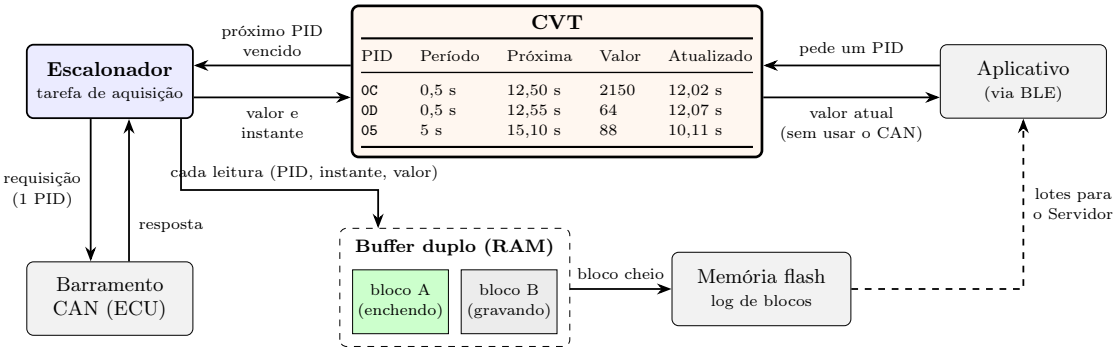



Figura 1: Fluxo da aquisição em torno da CVT.

A tarefa de aquisição decide qual próximo PID a ser lido. Quando a leitura termina, ela atualiza a CVT e coloca a leitura em um buffer duplo. Se isto enche o buffer, ele dispara a tarefa de escrita na flash, que fará o flush do bloco contendo várias leituras. Após isso, a tarefa de upload poderá executar, caso tenha conectividade.

A estrutura que será armazenada no buffer é a seguinte:

Tabela 3: Registro de leitura de PID (13 bytes).

| Campo | Tipo | Bytes | Descrição |
|---|---|---|---|
| `seq` | `uint32` | 4 | Número do registro, global e persistente |
| `boot_id` | `uint16` | 2 | Boot em que o registro foi gerado |
| `t_ms` | `uint32` | 4 | Milissegundos desde o boot |
| `tipo` | `uint8` | 1 | PID lido (`0x00` a `0xFE`) |
| `valor` | 2 bytes | 2 | Bytes da resposta; PIDs de 1 byte usam só o primeiro |
| Total | | 13 | |

A posição do GPS será tratada como um PID customizado, de valor 0xFF. Será gerado um registro por segundo, com a seguinte estrutura:

Tabela 4: Registro de posição GPS (23 bytes).

| Campo | Tipo | Bytes | Descrição |
|---|---|---|---|
| `seq` | `uint32` | 4 | Número do registro, global e persistente |
| `boot_id` | `uint16` | 2 | Boot em que o registro foi gerado |
| `t_ms` | `uint32` | 4 | Milissegundos desde o boot |
| `tipo` | `uint8` | 1 | Sempre `0xFF` |
| `lat` | `int32` | 4 | Latitude, em graus $\times 10^7$ |
| `lon` | `int32` | 4 | Longitude, em graus $\times 10^7$ |
| `alt` | `int32` | 4 | Altitude, em milímetros |
| Total | | 23 | |

Entendido como será o formato e o tamanho dos dados gerados pelo Dongle, é necessário modelar a comunicação para entender suas limitações.

### 2.4.3 Capacidade do Canal

Todos os dados vêm de uma única fonte: o barramento CAN. Já foi estabelecido na Subseção 1.1 que o modelo de requisição será sequencial, com um PID por requisição. Para entender como isto pode dar certo, vamos modelar o problema.

É importante relembrar que o que será modelado é um abstrato "Canal de Requisições" usado pelo Dongle, e não o barramento CAN em si. O CAN tem sua lógica de prioridade e N ECUs podem se comunicar entre si, sem ser afetadas por nossa utilização, que é de baixa prioridade. Além disso, é possível que as ECUs, comunicando-se entre si, utilizem-se da concorrência para aumentar o desempenho. No entando, o Dongle enxerga o CAN como um canal que só pode receber uma requisição por vez, e é isto que está sendo modelado.

Suponha que cada requisição demora um tempo $T_{\max}$ para ser atendida. A capacidade é de $\lambda_{\text{canal}} = \frac{1}{T_{\max}}$ requisições por segundo.

### 2.4.4 Utilização do Canal

Considere $n$ PIDs. Para cada PID $i$:

- $P_i$ é o período de leitura desejado;
- $T_i$ é o tempo de serviço da requisição, isto é, o intervalo entre o envio do pedido e a chegada da resposta, limitado pelo prazo $T_{\max}$;
- $d_i$ é o tamanho, em bytes, do valor retornado.

O Dongle aguarda cada resposta por no máximo $T_{\max}$, o prazo configurado. Como uma requisição sem resposta também ocupa o canal até o fim desse prazo, vale sempre $T_i \leq T_{\max}$, e substituir $T_i$ por $T_{\max}$ fornece o pior caso. Quando conveniente, supõe-se um tempo de serviço comum $T$ para todos os PIDs.

Numa janela de duração $W$, o PID $i$ é lido $W/P_i$ vezes, e cada leitura ocupa o barramento por $T_i$. O tempo ocupado por ele é, portanto, $W\,T_i/P_i$, e a fração do tempo total, $T_i/P_i$, independentemente do tamanho da janela. Somando as frações de todos os PIDs, obtém-se a fração do tempo em que o barramento está ocupado:

$$U = \sum_{i=1}^{n} \frac{T_i}{P_i}, \tag{2}$$

e uma configuração é considerada viável quando $U \leq U_{\max}$, com $U_{\max} < 1$ reservando folga para atrasos no próprio Dongle. Neste projeto, adota-se $U_{\max} = 0{,}8$.

Uma intuição é a seguinte. Suponha que um único PID seja amostrado a cada 100 ms, mas que cada requisição demore 200 ms para ser atendida. Quando chegar o momento da segunda leitura, a primeira ainda estará na metade. Como as requisições são atendidas uma de cada vez, a segunda só poderá começar depois que a primeira terminar, e assim sucessivamente. O PID será lido, na prática, a cada 200 ms, e não a cada 100 ms: metade das leituras pretendidas nunca acontecerá. Em termos da Equação 2, $U = 200/100 = 2$, ou seja, o PID precisaria de 200% do tempo disponível.

Se o período fosse de 200 ms, teríamos $U = 1$: o canal ficaria ocupado o tempo todo, sem nenhum intervalo livre. Funcionaria apenas em condições ideais, pois qualquer resposta um pouco mais lenta atrasaria todas as seguintes. Daí a necessidade de $U_{\max} < 1$: a folga absorve as variações no tempo de resposta e os atrasos do próprio Dongle.

### 2.4.5 Taxa de geração de dados

Seja $f_i = 1/P_i$ a frequência de leitura do PID $i$, $\rho_i$ a fração de requisições sem resposta e $f_G$ a frequência com que o aplicativo envia posições GPS. O número de registros gerados por segundo é

$$\Lambda_r = \sum_{i=1}^{n} f_i\,(1 - \rho_i) + f_G, \tag{3}$$

e a taxa de geração de dados, em bytes por segundo, é

$$R = s_L \sum_{i=1}^{n} f_i \,(1 - \rho_i) + s_G \, f_G. \tag{4}$$

### 2.4.6 Tempo até esgotar a memória

Seja $F$ a capacidade, em bytes, da partição usada para os registros. Se nenhum dado for enviado ao Servidor, a memória se esgota após

$$t_{\text{mem}} = \frac{F}{R} \tag{5}$$

segundos de coleta. A partir desse ponto, para continuar coletando, o Dongle precisa descartar os registros mais antigos.

A memória flash também tem vida útil limitada: cada setor suporta um número $E$ de ciclos de apagamento, tipicamente da ordem de $10^5$. Como o sistema de arquivos distribui as gravações por toda a partição, ela é inteiramente regravada a cada $F/R$ segundos de coleta, e a vida útil é

$$t_{\text{vida}} = \frac{E\,F}{R}. \tag{6}$$

### 2.4.7 Tempo até esgotar os contadores

O número de sequência é um inteiro de 32 bits, incrementado a cada registro. Ele se esgota após

$$t_{\text{seq}} = \frac{2^{32}}{\Lambda_r} \tag{7}$$

segundos de coleta.

Também é possível obter um limite que vale para qualquer configuração viável. Pela Equação 2, com tempo de serviço comum $T$, a viabilidade exige $\sum_i f_i \le U_{\max}/T$. Desprezando o GPS, segue que

$$t_{\text{seq}} \ge \frac{2^{32}\,T}{U_{\max}}. \tag{8}$$

Mesmo com uma ECU muito rápida, com $T = 10\,\text{ms}$, o número de sequência dura mais de 1,7 ano de coleta ininterrupta.

Os demais contadores têm limites próprios. O campo `t_ms`, de 32 bits em milissegundos, se esgota após $2^{32}\,\text{ms} \approx 49{,}7$ dias sem reinicialização; ao se aproximar desse valor, o Dongle inicia um novo boot. O campo `boot_id`, de 16 bits, suporta $2^{16} = 65\,536$ boots: com $b$ boots por dia, ele dura $65\,536/b$ dias, ou cerca de 36 anos para $b = 5$.

A Tabela 5 resume essas relações.

Tabela 5: Taxa de dados e limites de armazenamento e de contadores.

| Grandeza | Fórmula |
|---|---|
| Registros por segundo | $\Lambda_r = \sum_i f_i\,(1-\rho_i) + f_G$ |
| Taxa de dados | $R = s_L \sum_i f_i\,(1-\rho_i) + s_G\,f_G$ |
| Tempo até esgotar a memória | $t_{\text{mem}} = F/R$ |
| Vida útil da flash | $t_{\text{vida}} = E\,F/R$ |
| Tempo até esgotar o número de sequência | $t_{\text{seq}} = 2^{32}/\Lambda_r$ |
| Tempo até esgotar o `t_ms` | $2^{32}$ ms |
| Tempo até esgotar o `boot_id` | $2^{16}/b$ |

### 2.4.8 Parâmetros e comandos

Com a modelagem matemática pronta, fica claro que há decisões que precisam ser tomadas. Por exemplo, a listagem de PIDs já foi definida na Subsubseção 2.4.1. No entanto é necessário definir a frequência de coleta de cada um. Além disso, outras questões como: bitrate do CAN e tamanho do CAN ID, que serão utilizados na configuração de cada um; e o tempo máximo de espera antes do Timeout, ou $T_{\max}$.

O pequeno conjunto de parâmetros será configurado pelo aplicativo e gravado na memória não volátil (NVS), de modo que sobrevivem a reinicializações (Tabela 6). Os demais valores são constantes de projeto, fixadas no firmware (Tabela 7).

Tabela 6: Parâmetros configuráveis do Dongle.

| Parâmetro | Valores | Padrão | Ao alterar |
|---|---|---|---|
| Taxa do barramento | 250 kbps ou 500 kbps | 500 kbps | Reinicia o Dongle |
| Tamanho do CAN ID | 11 ou 29 bits | 11 bits | Reinicia o Dongle |
| Prazo de resposta $T_{\max}$ | múltiplos de 4 ms | 50 ms | Aplicado imediatamente |
| Lista de PIDs | até 16 PIDs do serviço 01 | — | Reconstrói a CVT |
| Período de cada PID | em milissegundos | — | Reconstrói a CVT |

A taxa do barramento e o tamanho do identificador não são independentes: a ISO 15765-4 prevê exatamente quatro combinações, e o ELM327 já as identifica como protocolos numerados. Por isso, os dois parâmetros são configurados juntos, por um único comando. Como reconfigurar o controlador CAN com as tarefas em execução é arriscado, o Dongle grava a nova configuração e reinicia, o que também gera um novo `boot_id` e separa, nos dados, o antes e o depois da mudança.

Toda alteração na lista de PIDs ou nos períodos é validada antes de ser aceita: o Dongle calcula a utilização pela Equação 2, com $T_i = T_{\max}$, e rejeita a configuração se $U > U_{\max}$.

### 2.4.9 Comandos AT

O aplicativo se comunica com o Dongle por comandos de texto no padrão do ELM327. A Tabela 8 lista os comandos suportados.

Tabela 7: Constantes de projeto.

| Constante | Valor | Justificativa |
|---|---|---|
| Utilização máxima $U_{\max}$ | 0,8 | Folga para atrasos (Equação 2) |
| Tamanho de cada bloco do buffer | 1 KB | Equilíbrio entre RAM e número de gravações |
| Intervalo máximo de flush | 10 s | Limita a perda num desligamento brusco |
| Capacidade da CVT | 16 PIDs | Folga sobre os 7 PIDs previstos |

Tabela 8: Comandos AT suportados pelo Dongle.

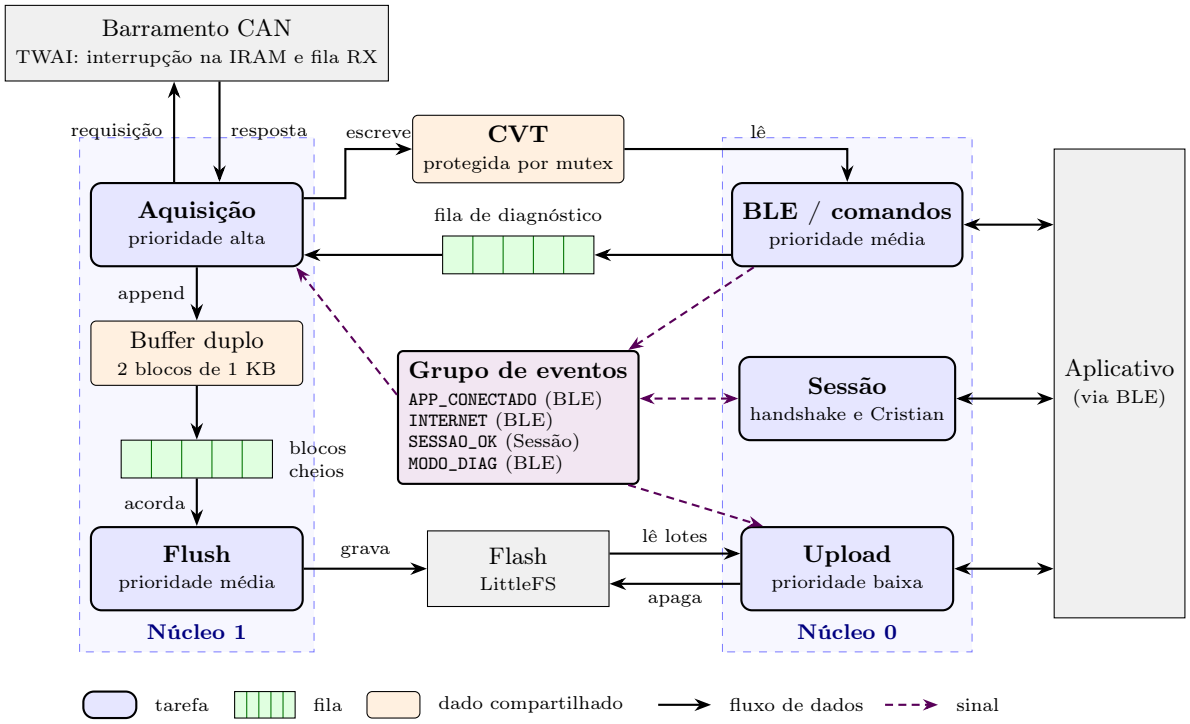
| Comando | Origem | Função |
|---|---|---|
| `AT Z` | ELM327 | Reinicia o Dongle |
| `AT I` | ELM327 | Identificação do dispositivo |
| `AT E0`, `AT E1` | ELM327 | Desativa ou ativa o eco dos comandos |
| `AT SP h` | ELM327 | Define o protocolo (Tabela 9) |
| `AT DP` | ELM327 | Informa o protocolo em uso |
| `AT ST hh` | ELM327 | Define $T_{\max} =$ `hh` $\times$ 4 ms |
| `AT RV` | ELM327 | Lê a tensão da bateria |
| `AT#PID pp mmmm` | Projeto | Adiciona ou altera o PID `pp` com período `mmmm` ms |
| `AT#DEL pp` | Projeto | Remove o PID `pp` |
| `AT#LIST` | Projeto | Lista os PIDs, os períodos e a utilização |
| `AT#DIAG b` | Projeto | Ativa (`1`) ou desativa (`0`) o modo diagnóstico |
| `AT#NET b` | Projeto | Informa se o aplicativo tem acesso à Internet |
| `AT#GPS lat lon alt` | Projeto | Envia a posição atual (graus $\times 10^7$ e mm) |

Tabela 9: Protocolos CAN aceitos pelo comando `AT SP`.

| Código | Tamanho do ID | Taxa |
|---|---|---|
| `6` | 11 bits | 500 kbps |
| `7` | 29 bits | 500 kbps |
| `8` | 11 bits | 250 kbps |
| `9` | 29 bits | 250 kbps |

Um comando com PID ou período inválido, ou que tornaria a configuração inviável, é respondido com `?`, como no ELM327. Fora do modo diagnóstico, requisições de PIDs presentes na CVT são respondidas a partir da tabela, sem acessar o barramento.

### 2.4.10 Arquitetura Final




## 2.5 Procedimento

Para configurar o Dongle em um veículo, é preciso definir o prazo de resposta $T_{\max}$ e o período $P_i$ de cada PID.

### 2.5.1 Definição de $T_{\max}$

Com o motor ligado, o Dongle envia 20 requisições de cada PID e mede o tempo até cada resposta. Adota-se como prazo o maior percentil 99 entre os PIDs, com uma margem de 30%:

$$T_{\max} = 1{,}3 \cdot \max_i T_i^{(99)}. \tag{9}$$

### 2.5.2 Definição dos períodos

Cada período é definido como um múltiplo de $T_{\max}$, $P_i = k_i\, T_{\max}$, com $k_i$ menor para os PIDs que precisam de leituras mais frequentes (Tabela 10). Assim, a utilização $U = \sum_i T_{\max}/P_i = \sum_i 1/k_i$ não depende de $T_{\max}$: com os valores propostos, $U \approx 0{,}78$, próximo de $U_{\max} = 0{,}8$, em qualquer veículo. Os PIDs mais frequentes usam $k_i = 5$, o menor multiplicador inteiro que mantém $U \leq U_{\max}$: com $k_i = 4$, a utilização ultrapassaria 0,9.

Tabela 10: Períodos propostos em função de $T_{\max}$.

| PID | Grandeza | $P_i$ |
|---|---|---|
| `0C` | Rotação do motor | $5\,T_{\max}$ |
| `0D` | Velocidade do veículo | $5\,T_{\max}$ |
| `49` | Posição do pedal | $5\,T_{\max}$ |
| `04` | Carga calculada do motor | $12\,T_{\max}$ |
| `10` | Vazão mássica de ar (MAF) | $12\,T_{\max}$ |
| `05` | Temperatura do líquido | $100\,T_{\max}$ |
| `2F` | Nível de combustível | $200\,T_{\max}$ |

### 2.5.3 Valores resultantes

A Tabela 11 aplica as equações anteriores a três valores de $T_{\max}$, considerando uma posição GPS por segundo e uma partição de 2 MB.

Tabela 11: Períodos e limites para diferentes valores de $T_{\max}$.

| | $T_{\max} = 50\,\text{ms}$ | $T_{\max} = 100\,\text{ms}$ | $T_{\max} = 200\,\text{ms}$ |
|---|---|---|---|
| Período de `0C`, `0D`, `49` | 0,25 s | 0,5 s | 1 s |
| Período de `04`, `10` | 0,6 s | 1,2 s | 2,4 s |
| Período de `05` | 5 s | 10 s | 20 s |
| Período de `2F` | 10 s | 20 s | 40 s |
| Utilização $U$ | 0,78 | 0,78 | 0,78 |
| Registros por segundo $\Lambda_r$ | 16,6 | 8,8 | 4,9 |
| Taxa de dados $R$ | 226 B/s | 125 B/s | 74 B/s |
| Dados por hora | 814 kB | 449 kB | 266 kB |
| Tempo até esgotar a memória $t_{\text{mem}}$ | 2,6 h | 4,7 h | 7,9 h |
| Tempo até esgotar o `seq` $t_{\text{seq}}$ | 8 anos | 15 anos | 28 anos |

Os períodos da Tabela 11 são conservadores, pois supõem que toda requisição consome o prazo completo $T_{\max}$. Na prática, a ECU pode responder bem antes, mas como os períodos continuam fixos, o canal pode passar boa parte do tempo ocioso. Pode ser interessante pensarmos em como mudar para que ele fique adaptativo.

## 2.6 Possíveis futuras otimizações

Por fim, algumas otimizações ficam como possíveis evoluções do projeto.

### 2.6.1 Otimizando o tamanho das leituras

A mais direta é reduzir o tamanho dos registros: em vez de repetir `seq`, `boot_id` e `t_ms` em cada leitura, esses campos poderiam ser gravados uma única vez no cabeçalho de cada bloco. O cabeçalho guardaria o instante de início do bloco, e cada registro guardaria apenas a diferença de tempo em relação ao início do bloco. Como o bloco dura no máximo 10 segundos antes de ser armazenado, a diferença

caberia em 2 bytes, em milissegundos. O registro de leitura, com o PID, o valor e a diferença de tempo, cairia de 13 para 5 bytes, reduzindo a taxa de dados em cerca de 60% e aumentando na mesma proporção a autonomia da memória. O custo é uma lógica de gravação e recuperação mais complexa.

### 2.6.2 Uma otimização otimista

No comportamento atual, o reagendamento é calculado a partir de $T_{\max}$. Sempre que a ECU responde antes do prazo o escalonador termina as leituras vencidas e permanece parado até o próximo vencimento, gerando ociosidade. Para evitar esse desperdício, cada linha da CVT pode passar a ter mais duas colunas: um intervalo mínimo $P_i^{\min}$ entre leituras do mesmo PID e o instante da última leitura realizada.

A regra do escalonador passaria a ser a seguinte: se houver PIDs vencidos, atende-se o vencido há mais tempo, como antes; se não houver, em vez de esperar, lê-se antecipadamente o PID com o vencimento mais próximo entre aqueles cuja última leitura ocorreu há pelo menos $P_i^{\min}$; somente se nenhum PID atender a essa condição o escalonador aguarda. Após uma leitura antecipada, o próximo vencimento é contado a partir dela. Dessa forma, os períodos $P_i$ passam a ser intervalos máximos, que continuam garantidos, enquanto $P_i^{\min}$ limita a frequência de cada PID.

A utilização do canal aumenta sempre que a ECU responde rapidamente, e a taxa de dados, embora variável, fica limitada a $s_L \sum_i 1/P_i^{\min} + s_G\, f_G$, valor que passa a ser usado no cálculo da autonomia da memória.

### 2.6.3 Estimativas com as otimizações

Esta seção refaz as estimativas supondo as duas otimizações: registros agrupados em blocos e leitura antecipada com intervalo mínimo por PID.

### 2.6.4 Formato em blocos

O número de sequência, o `boot_id` e o instante de início passam a ser gravados uma única vez, no cabeçalho de cada bloco. Cada registro guarda apenas o tempo decorrido desde o início do bloco, em milissegundos. Como um bloco é fechado em no máximo $T_{\text{flush}} = 10\,\text{s}$, esse valor cabe em 2 bytes (Tabela 12).

Tabela 12: Estruturas do formato em blocos.

| Estrutura | Campos | Bytes |
|---|---|---|
| Cabeçalho (um por bloco) | `seq` (4), `boot_id` (2), `t_ms` inicial (4), tamanho (2) | $H = 12$ |
| *Registros (vários por bloco)* | | |
| Leitura de PID | `tipo` (1), $\Delta t$ (2), `valor` (2) | $s_L = 5$ |
| Posição GPS | `tipo` (1), $\Delta t$ (2), `lat`, `lon`, `alt` (12) | $s_G = 15$ |

### 2.6.5 Intervalo mínimo

Cada PID passa a ter, além do período $P_i$, um intervalo mínimo $P_i^{\min}$, que por padrão é igual a $P_i$ e pode ser reduzido, respeitando um piso absoluto de 100 ms. Para os PIDs que se beneficiam de leituras mais frequentes, propõe-se $P_i^{\min} = \max(P_i/2,\ 100\,\text{ms})$; para os demais, mantém-se $P_i^{\min} = P_i$, pois leituras adicionais só repetiriam valores (Tabela 13).

Tabela 13: Período e intervalo mínimo de cada PID.

| PID | Grandeza | $P_i$ | $P_i^{\min}$ |
|---|---|---|---|
| `0C` | Rotação do motor | $5\,T_{\max}$ | $\max(P_i/2,\ 100\,\text{ms})$ |
| `0D` | Velocidade do veículo | $5\,T_{\max}$ | $\max(P_i/2,\ 100\,\text{ms})$ |
| `49` | Posição do pedal | $5\,T_{\max}$ | $\max(P_i/2,\ 100\,\text{ms})$ |
| `04` | Carga calculada do motor | $12\,T_{\max}$ | $\max(P_i/2,\ 100\,\text{ms})$ |
| `10` | Vazão mássica de ar (MAF) | $12\,T_{\max}$ | $\max(P_i/2,\ 100\,\text{ms})$ |
| `05` | Temperatura do líquido | $100\,T_{\max}$ | $P_i$ |
| `2F` | Nível de combustível | $200\,T_{\max}$ | $P_i$ |

Com a leitura antecipada, o número de leituras por segundo deixa de ser fixo e passa a variar entre um valor nominal, quando a ECU é lenta e cada PID é lido no seu período, e um valor máximo, quando a ECU é rápida e cada PID é lido no seu intervalo mínimo:

$$\Lambda^{\text{nom}} = \sum_{i=1}^{n} \frac{1}{P_i} \;\le\; \Lambda \;\le\; \Lambda^{\max} = \sum_{i=1}^{n} \frac{1}{P_i^{\min}}. \tag{10}$$

### 2.6.6 Equações

A taxa de dados dos registros é $R_{\text{reg}} = s_L\,\Lambda + s_G\,f_G$. Como cada bloco tem $S = 1024\,\text{B}$ e é fechado ao encher ou após $T_{\text{flush}}$, o número de blocos por segundo e a taxa total são

$$b = \max\!\left(\frac{R_{\text{reg}}}{S-H},\ \frac{1}{T_{\text{flush}}}\right), \qquad R = R_{\text{reg}} + H\,b. \tag{11}$$

O tempo até esgotar a memória continua sendo $t_{\text{mem}} = F/R$. Já o número de sequência passa a ser incrementado por bloco, e não por registro:

$$t_{\text{seq}} = \frac{2^{32}}{b}. \tag{12}$$

Como cada bloco reúne dezenas de registros, $b$ é muito menor que o número de leituras por segundo: nos cenários a seguir, ele fica entre 0,1 e 0,17 bloco por segundo, e $t_{\text{seq}}$ supera 800 anos. O `boot_id` não muda: continua limitado a $2^{16}$ boots.

### 2.6.7 Resultados

A Tabela 14 aplica as equações aos mesmos valores de $T_{\max}$ da Tabela 11, com uma posição GPS por segundo e uma partição de 2 MB. A última linha repete,

Tabela 14: Estimativas com registros em blocos e intervalo mínimo.

| | $T_{\max} = 50\,\text{ms}$ | $T_{\max} = 100\,\text{ms}$ | $T_{\max} = 200\,\text{ms}$ |
|---|---|---|---|
| $P_i$ / $P_i^{\min}$ de `0C`, `0D`, `49` | 250 / 125 ms | 500 / 250 ms | 1000 / 500 ms |
| $P_i$ / $P_i^{\min}$ de `04`, `10` | 600 / 300 ms | 1200 / 600 ms | 2400 / 1200 ms |
| Leituras por segundo, nominal / máximo | 15,6 / 31,0 | 7,8 / 15,5 | 3,9 / 7,7 |
| Taxa de dados $R$, nominal / máximo | 94 / 172 B/s | 55 / 94 B/s | 36 / 55 B/s |
| Dados por hora, nominal / máximo | 340 / 619 kB | 199 / 337 kB | 129 / 198 kB |
| $t_{\text{mem}}$, nominal / pior caso | 6,2 / 3,4 h | 10,5 / 6,2 h | 16,3 / 10,6 h |
| $t_{\text{seq}}$ | $> 800$ anos | $> 1000$ anos | $> 1000$ anos |
| $t_{\text{mem}}$ no formato anterior | 2,6 h | 4,7 h | 7,9 h |

para comparação, a autonomia do formato anterior, com 13 bytes por registro e sem leitura antecipada.

A comparação mostra que as duas otimizações se compensam com folga. A leitura antecipada pode dobrar a frequência dos PIDs mais importantes e, no pior caso, quase dobrar o volume de dados; ainda assim, graças ao formato em blocos, a autonomia no pior caso continua maior que a do formato anterior sem leitura antecipada. O número máximo de leituras por segundo só é atingido se a ECU responder rápido o suficiente: com $\Lambda^{\max} = 31$ leituras por segundo, por exemplo, é preciso um tempo de resposta médio inferior a 32 ms.